

PROJETO E ANÁLISE DE ALGORITMOS – 01A – 2022.2

[Página inicial](#)

[Meus cursos](#)

[PROJETO E ANÁLISE DE ALGORITMOS – 01A – 2022.2](#)

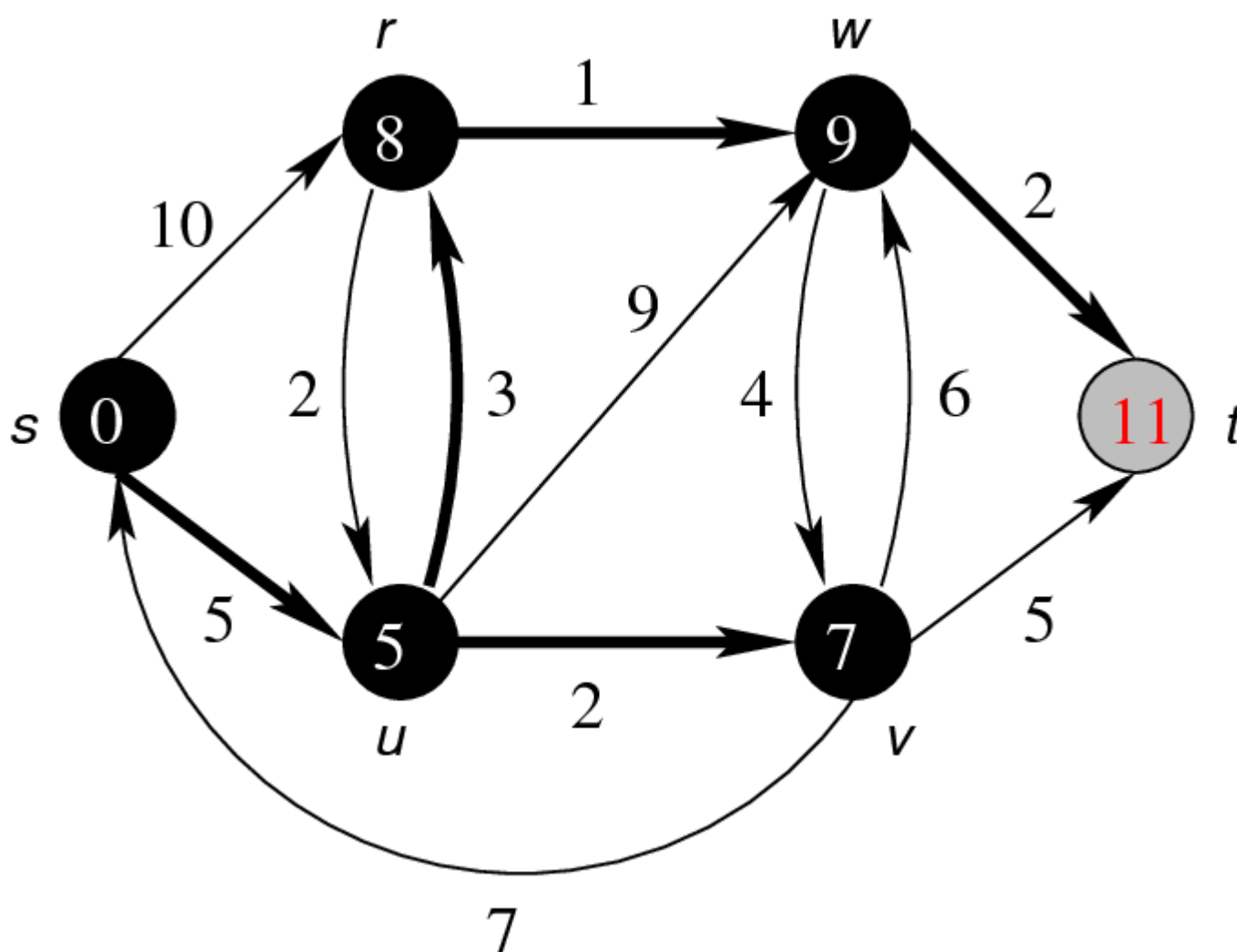
[Tópico 9](#)

[Notas de aula: caminhos mais curtos com mesma origem](#)

Problema dos Caminhos Mínimos com Mesma Origem: Seja (G) um grafo direcionado com pesos nas arestas, e (s) um vértice de (G) . Encontrar os caminhos mínimos (menor soma dos pesos das arestas) de (s) para todos os outros vértices.

Para recuperar os caminhos mínimos, guardamos o predecessor de cada vértice. Isso produz uma árvore de caminhos mínimos com raiz em (s) .

O problema de encontrar um caminho mínimo entre par de vértices (s) e (t) é caso particular do problema de encontrar caminhos mínimos com mesma origem. Embora seja um caso particular tem a mesma complexidade, pois no pior caso teremos calculado os caminhos mínimos dos outros vértices antes de obter o caminho mínimo até (t) .



Armazenamos em $(w[u,v])$ o peso de cada aresta $((u,v))$.

Denotamos por $(dist(v))$ o tamanho do caminho mínimo de (s) até (v) .

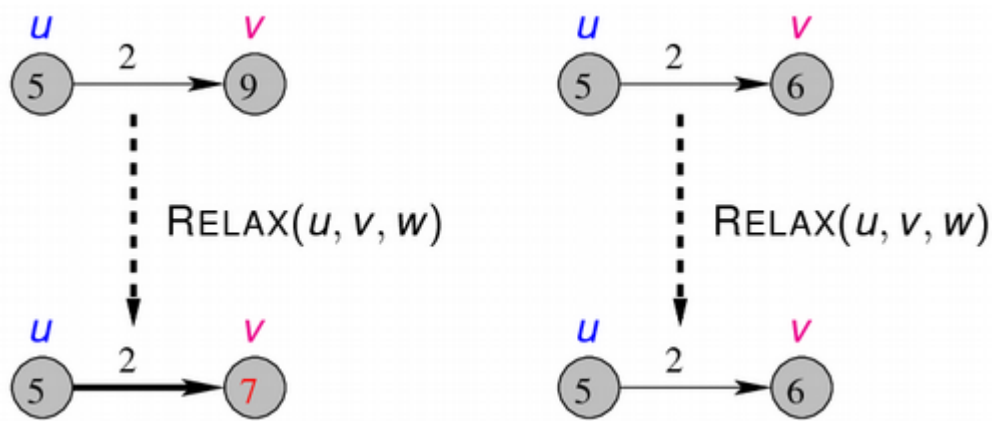
Para cada vértice (v) , armazenamos em $(d[v])$ um limite superior para $(dist(v))$, ou seja, os algoritmos devem manter $(dist(v) \leq d[v])$.

$(d[v])$ é inicializado com $(-\infty)$, exceto $(d[s])$ que é inicializado com (0) .

Para cada vértice (v) , armazenamos em $(\pi[v])$ o predecessor de (v) na árvore de caminhos mínimos.

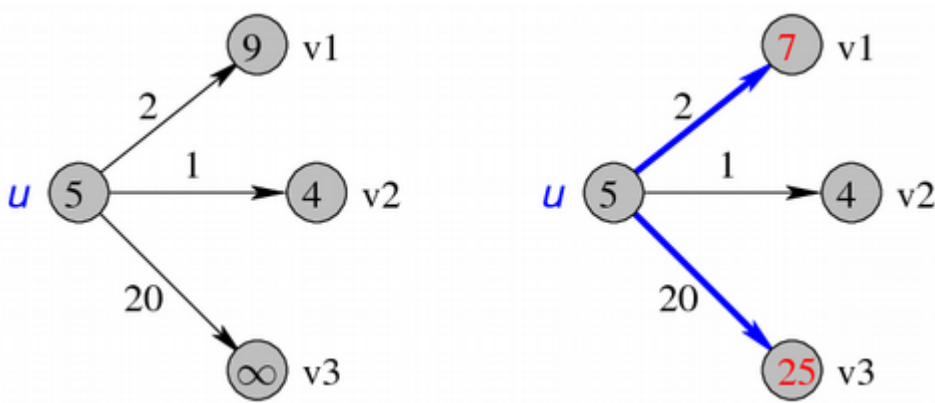
Operação de relaxação de arestas

A operação RELAX tenta melhorar a estimativa em $(d[v])$.



```
RELAX(u, v):
  se d[v] > d[u] + w[u, v]
    d[v] <- d[u] + w[u, v]
    pi[v] <- u
```

Relaxação dos vizinhos de um vértice u :



Veremos três algoritmos baseados em relaxação:

- G é acíclico: relaxação em ordem topológica.
- Não tem pesos negativos: algoritmo de Dijkstra.
- Tem pesos negativos, mas não tem ciclos negativos: algoritmo de Bellman-Ford.

Propriedades

Desigualdade triangular: Para toda aresta (u, v) temos que $\text{dist}(v) \leq \text{dist}(u) + w[u, v]$.

Um caminho mínimo de s até u , juntamente com a aresta (u, v) , forma um caminho de s até v custo $\text{dist}(u) + w[u, v]$. Portanto, um caminho mínimo de s até v não pode custar mais do que isso.

Se não existir caminho de s até v , também não haverá caminho de s até u . Neste caso ambos $\text{dist}(s)$ e $\text{dist}(v)$ valem ∞ e a propriedade vale.

Propriedade de limite superior: Após qualquer sequência de operações RELAX temos que $\text{dist}(u) \leq d[u]$ para vértice u .

Por indução no número de relaxamentos.

Base: na inicialização $\text{dist}(s)=0$ e $d[s]=0$ e $d[u]=\infty$ para todo vértice $u \neq s$. Temos que $\text{dist}(s)=0$ quando s não pertence a um ciclo negativo, e $\text{dist}(s)=-\infty$ caso contrário. Neste dois caso e para todos os outros a propriedade vale.

Passo indutivo: suponha que a propriedade vale antes da execução de uma operação RELAX na aresta (u, v) . Em a operação, se houver mudança no valor $d[v]$ teremos

$$d[v] = d[u] + w[u, v] \geq \text{dist}(u) + w[u, v] \geq \text{dist}(v)$$

(pela desigualdade triangular.)

Corolário: Quando $d[u]$ atingir $\text{dist}(u)$ seu valor não mudará mais.

Vimos que $d[u]$ não pode ser menor que $\text{dist}(u)$, e além disso a operação RELAX nunca aumenta o valor de $d[u]$.

Corolário: Se não existir caminho de s até u , então $d[u]$ terá sempre valor ∞ .

Como $\text{dist}(u) = \infty \leq d[u]$, o valor ∞ que $d[u]$ recebeu na inicialização será mantido.

Lemma (desigualdade triangular após RELAX): Após executar RELAX (u, v) , teremos $d[v] \leq d[u] + w[u, v]$.

Se $(d[v] \leq d[u] + w[u,v])$ antes da relaxação, então $(d[v])$ não será modificado. Caso contrário, a relaxação $(d[v] = d[u] + w[u,v])$.

Lema (subestrutura ótima de caminhos mínimos): Se $\langle (v_1, v_2, \dots, v_k) \rangle$ é um caminho mínimo de (v_1) a (v_k) então, para todo $(1 \leq i \leq j \leq k)$ temos que $\langle (v_i, \dots, v_j) \rangle$ é um caminho mínimo de (v_i) a (v_j) .

Se existir caminho de menor custo entre (v_i) e (v_j) , podemos construir um caminho mais barato entre (v_1) a (v_k) , contradizendo a premissa de que $\langle (v_1, v_2, \dots, v_k) \rangle$ é um caminho mínimo.

Lema (propriedade de convergência): Se existe caminho mínimo de (s) até (v) passando pela aresta $((u,v), (d[u] = \text{dist}(u)))$ antes da chamada $(\text{RELAX}(u,v))$, então teremos $(d[v] = \text{dist}(v))$ após a chamada.

Após a chamada $(\text{RELAX}(u,v))$, teremos
$$d[v] \leq d[u] + w[u,v] \text{ (desigualdade triângulo após RELAX,)} \\ \&= \text{dist}(u) + w[u,v] \text{ (premissa,)} \\ \&= \text{dist}(v) \text{ (subestrutura ótima de caminhos mínimos)} \\ \end{align} $$$ Como $(\text{dist}(v) \leq d[v])$ pela propriedade de limite superior, concluímos que $(d[v] = \text{dist}(v))$.$$

Lema (propriedade de relaxamento de caminho): Para qualquer caminho mínimo $\langle (s, v_1, v_2, \dots, v_k) \rangle$ de (s) a (v_k) , se aplicarmos a sequência de relaxações $(\text{RELAX}(s,v_1))$, $(\text{RELAX}(v_1,v_2))$, ..., $(\text{RELAX}(v_{k-1},v_k))$, nesta ordem, mesmo intercalando com outras relaxações, teremos $(d[v_i] = \text{dist}(v_i))$ para todo $(i=1,..,k)$.

Por indução em (i) .

Base: $(i=1)$. Como existe caminho mínimo até (v_k) , então (s) não está contido em um ciclo negativo. Portanto $(\text{dist}(s) = 0 = d[s])$. Pela propriedade de convergência, teremos $(d[v_1] = \text{dist}(v_1))$ após a chamada $(\text{RELAX}(s,v_1))$.

Passo indutivo: por hipótese de indução temos $(d[v_{i-1}] = \text{dist}(v_{i-1}))$. Então pela propriedade de convergência teremos $(d[v_i] = \text{dist}(v_i))$ após a chamada $(\text{RELAX}(v_{i-1},v_i))$.

Lema (árvore de caminhos mínimos): Os predecessores armazenados em $\text{pi}[]$ formam uma árvore de caminhos mínimos (s) para todos os outros vértices.

Por indução no número de relaxações que fazem $(d[u] = \text{dist}(u))$ em todos os vértices.

Base: na inicialização os vértices não possuem predecessores, e a árvore de caminhos mínimos contém apenas o (s) .

Passo indutivo: assuma por hipótese de indução que todos os vértices (u) com $(d[u] = \text{dist}(u))$, e seus predecessores formam uma árvore de caminhos mínimos (ou seja, a soma dos pesos das arestas seguindo os predecessores é igual $(\text{dist}(u))$). Considere o instante em que o próximo vértice (v) ficará com $(d[v] = \text{dist}(v))$ após a relaxação da $((u,v))$.

O vértice (u) tem $(d[u]=\text{dist}(u))$, pois caso contrário teríamos um caminho mínimo até (v) com custo $(\text{dist}(u) + d[u] + w[u,v] > \text{dist}(u) + w[u,v])$, o que não é possível pois um caminho mínimo até (u) juntamente com a aresta $((u,v))$ forma um caminho válido de (s) até (v) .

Como o predecessor de (v) é o vértice (u) , e (u) está na árvore de caminhos mínimos, temos que a soma das arestas de (s) até (v) seguindo os predecessores vale $(d[u]+w[u,v] = d[v] = \text{dist}(v))$.

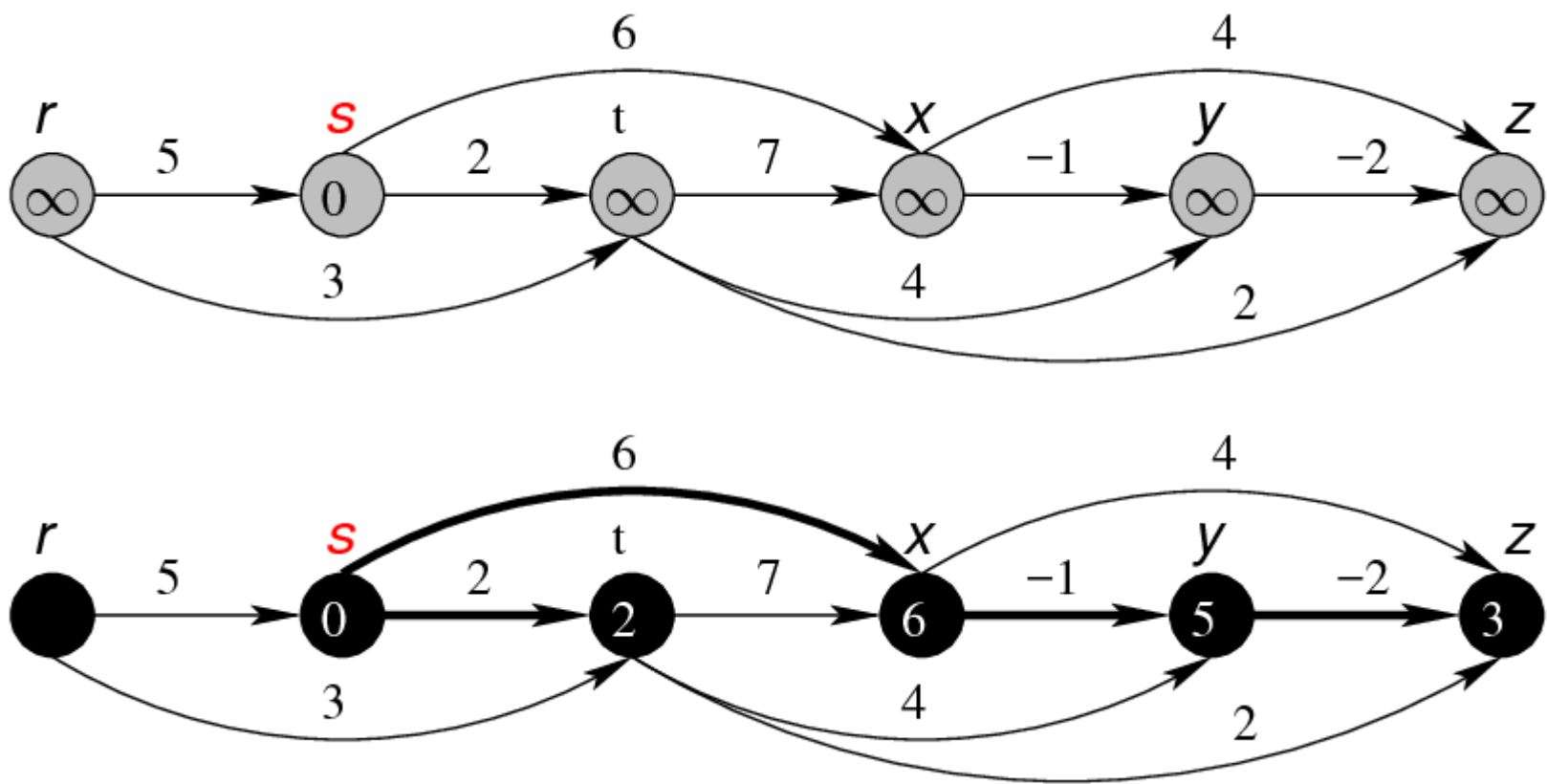
Caminhos mínimos em grafos direcionados acíclicos (DAGs)

Quando o grafo é acíclico podemos obter uma ordenação topológica dos vértices, ou seja, podemos ordenar os vértices de modo que toda aresta vai da esquerda para a direita.

Portanto, em todo caminho no grafo os vértices ocorrem em ordem topológica. Em particular isso vale para todo cc mínimo.

Assim, se relaxarmos as arestas que saem de cada vértice, seguindo uma ordem topológica dos vértices, atenderemos condições da Propriedade de Relaxamento de Caminhos, resultando em $(d[u] = \text{dist}(u))$ para todo vértice (u) .

Exemplo: resposta completa nos slides.



INICIALIZAÇÃO(s):

para todo vértice v, faça $d[v] \leftarrow \infty$, $\pi[v] \leftarrow \text{NULO}$
 $d[s] \leftarrow 0$

DAG-SHORTEST-PATH(s):

INICIALIZAÇÃO(s)	$O(V)$
encontre ordenação topológica para os vértices	$O(V+E)$
para cada vértice u em ordem topológica	$O(V)$
para cada v em Adj[u]	$O(V+E)$
RELAX(u,v)	$O(E)$

Complexidade: $\mathcal{O}(V+E)$.

Caminhos mínimos em grafos com ciclos e pesos não negativos: Algoritmo de Dijkstra

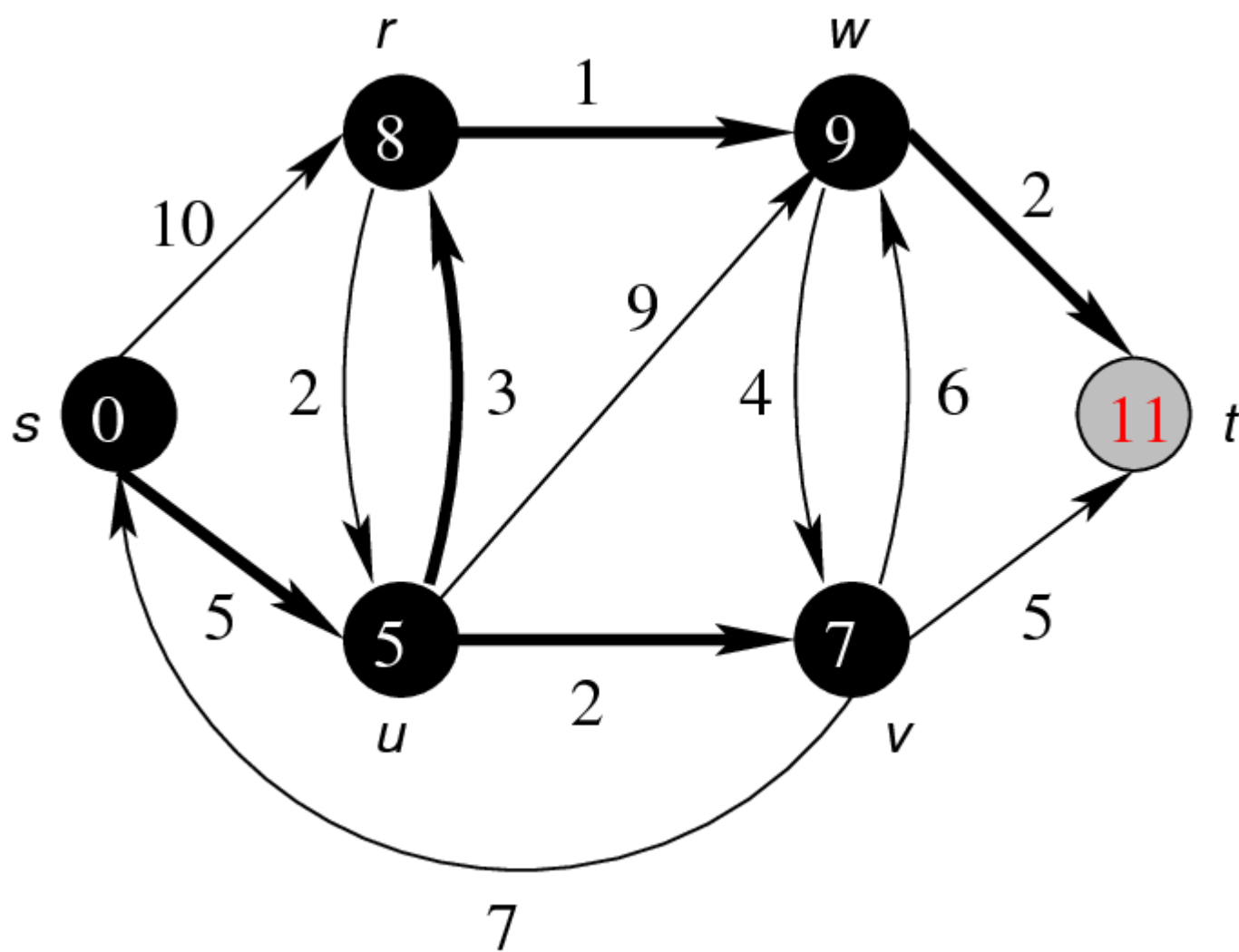
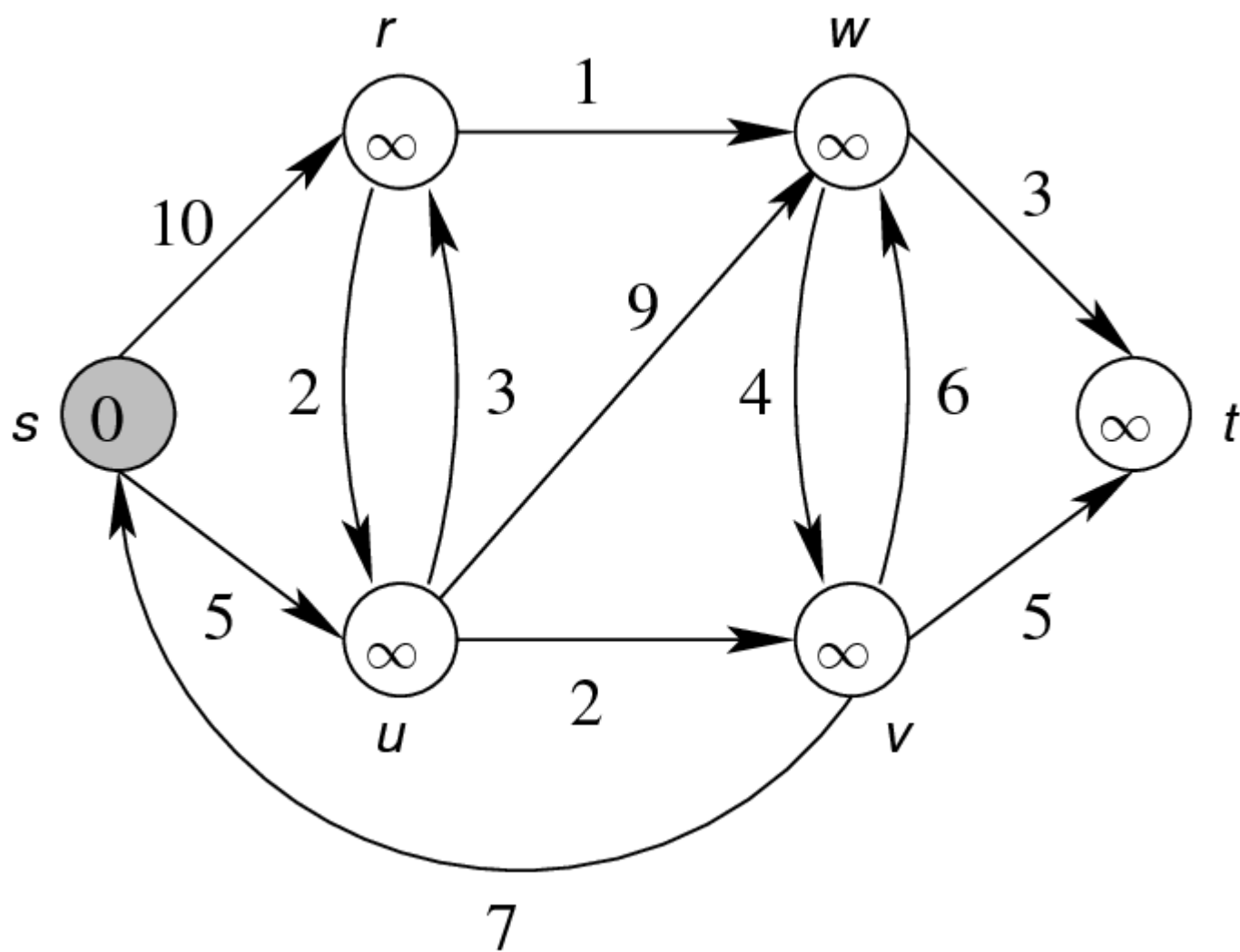
Quando o grafo tem ciclos, mas **não tem aresta com peso negativo**, podemos aplicar o algoritmo de Dijkstra.

Funciona de forma similar ao Prim.

Em cada iteração:

- Mantém em S os vértices que estão na árvore de caminhos mínimos encontrada até então.
- Encontra vértice u fora de S que esteja mais próximo dos vértices em S , e insere u em S .
- Atualiza $d[u]$ e $\pi[u]$.

Exemplo: resposta completa nos slides.



```

DIJKSTRA(s):
  INICIALIZAÇÃO(s)                                0(V)
  S <- {}                                           0(1)
  Q <- min-heap inicializadas c/ vértices c/ chaves d[] 0(V)
  enquanto Q não está vazia                        0(V)
    u <- EXTRACT-MIN(Q)                            0(V lg V)
    S <- S U {u}                                    0(V)
    para cada vértice v em Adj[u] e fora de S       0(V+E)
      RELAX(u,v)                                    0(E)
      DECREASE-KEY(Q,v,d[v])                        0(E lg V)

```

Complexidade: $\mathcal{O}((V+E) \lg V)$.

Grafo denso: $\mathcal{O}(V^2 \lg V)$. Grafo esparso: $\mathcal{O}(V \lg V)$.

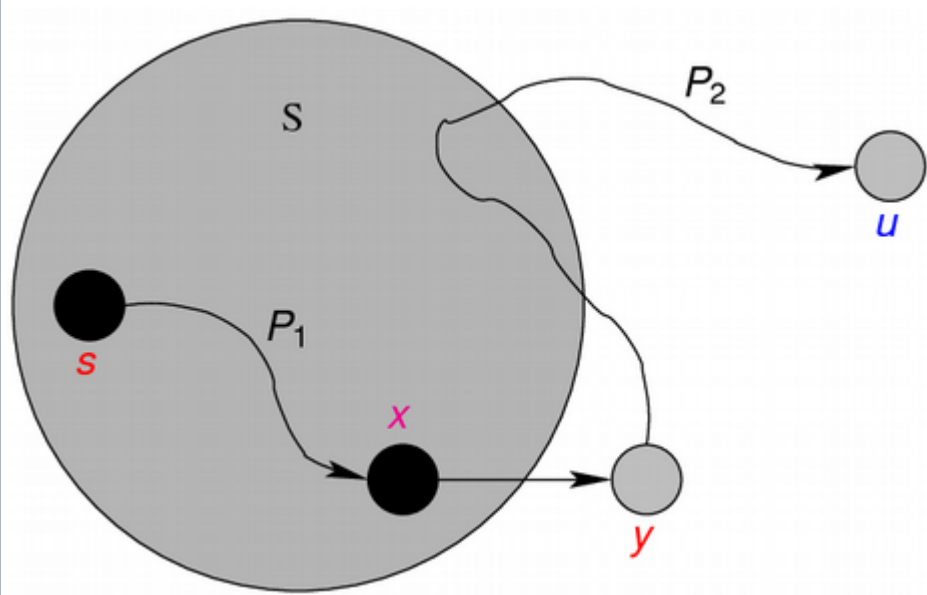
Usando heap de Fibonacci podemos fazer o DECREASE-KEY em $\mathcal{O}(1)$, porém as constantes são tão altas que na melhor usar heap binária.

DIJKSTRA(s):	
INICIALIZAÇÃO(s)	0(V)
S <- {}	0(1)
Q <- min-heap inicializadas c/ vértices c/ chaves d[]	0(V)
enquanto Q não está vazia	0(V)
u <- EXTRACT-MIN(Q)	0(V lg V)
S <- S U {u}	0(V)
para cada vértice v em Adj[u] e fora de S	0(V+E)
RELAX(u,v)	0(E)
DECREASE-KEY(Q,v,d[v])	----> 0(E)

Complexidade: $\backslash(O(V \backslash \log V + E)\backslash)$.
Grafo denso: $\backslash(O(V^2)\backslash)$. Grafo esparsos: $\backslash(O(V \backslash \log V)\backslash)$.

Corretude: Como ao final do algoritmo todos os vértices serão inseridos em $\backslash(S\backslash)$, e o $\backslash(d\backslash)$ dos vértices em $\backslash(S\backslash)$ r modificados, basta mostrar que todo vértice $\backslash(u\backslash)$ escolhido para entrar em $\backslash(S\backslash)$ tem $\backslash(d[u] = \text{dist}(u)\backslash)$.

Por indução no número de vértices em $\backslash(S\backslash)$.
Base: na inicialização $\backslash(S = \{s\}\backslash)$, $\backslash(d[s]=0\backslash)$, e $\backslash(\text{dist}(s)=0\backslash)$ pois não existe ciclo negativo passando por $\backslash(s\backslash)$.
Passo indutivo:
Considere o instante em que $\backslash(u \notin S\backslash)$ com menor $\backslash(d[u]\backslash)$ é escolhido para entrar em $\backslash(S\backslash)$. Assuma por coi que $\backslash(d[u] > \text{dist}(u)\backslash)$.
Seja $\backslash(P\backslash)$ um caminho mínimo qualquer de $\backslash(s\backslash)$ até $\backslash(u\backslash)$, $\backslash(y\backslash)$ o lo vértice de $\backslash(P\backslash)$ fora de $\backslash(S\backslash)$ e $\backslash(x\backslash)$ o pre de $\backslash(y\backslash)$ em $\backslash(P\backslash)$.



Pela Subestrutura Ótima de Caminhos Mínimos, como $\backslash(P_1 \cup \{(x,y)\}\backslash)$ é um caminho de $\backslash(s\backslash)$ até $\backslash(y\backslash)$ com um caminho mínimo de $\backslash(s\backslash)$ até $\backslash(u\backslash)$, concluímos que $\backslash(P_1 \cup \{(x,y)\}\backslash)$ é um caminho mínimo de $\backslash(s\backslash)$ at

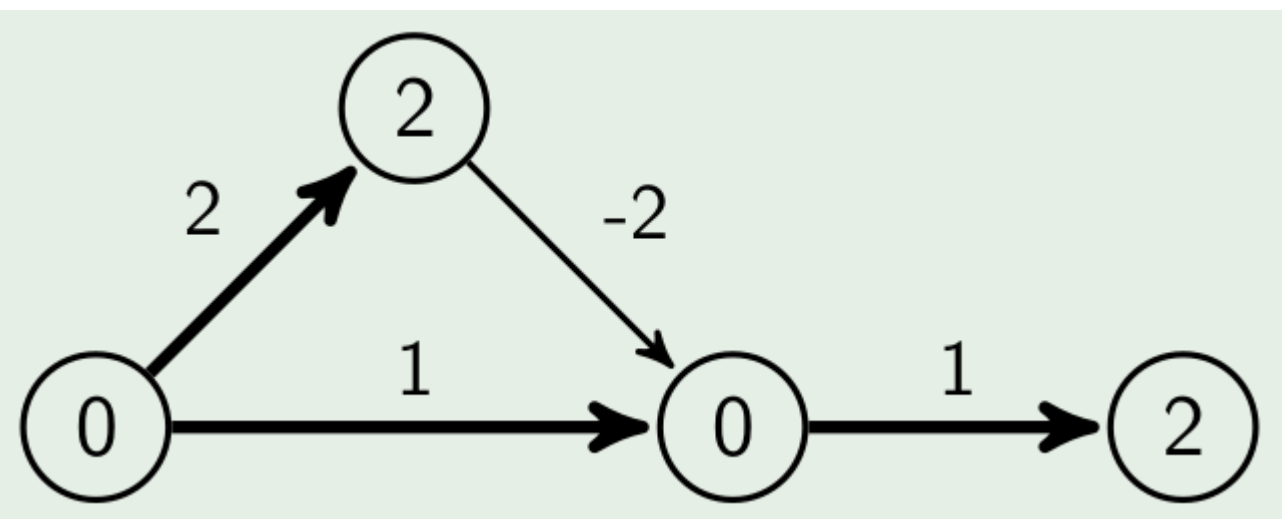
Por hipótese de indução o vértice $\backslash(x\backslash)$ tinha $\backslash(d[x] = \text{dist}(x)\backslash)$ quando entrou em $\backslash(S\backslash)$, e em seguida foi relaxada $\backslash((x,y)\backslash)$, tornando assim $\backslash(d[y] = \text{dist}(y)\backslash)$ pela Propriedade de Convergência.

Como os pesos são não negativos, o custo de $\backslash(P_2\backslash)$ é não negativo. Pela Subestrutura Ótima de Caminhos Mínir $\backslash(\text{dist}(u)\backslash)$ vale $\backslash(\text{dist}(y)\backslash)$ mais o custo de $\backslash(P_2\backslash)$. Portanto, $\backslash(\text{dist}(y) \leq \text{dist}(u)\backslash)$.

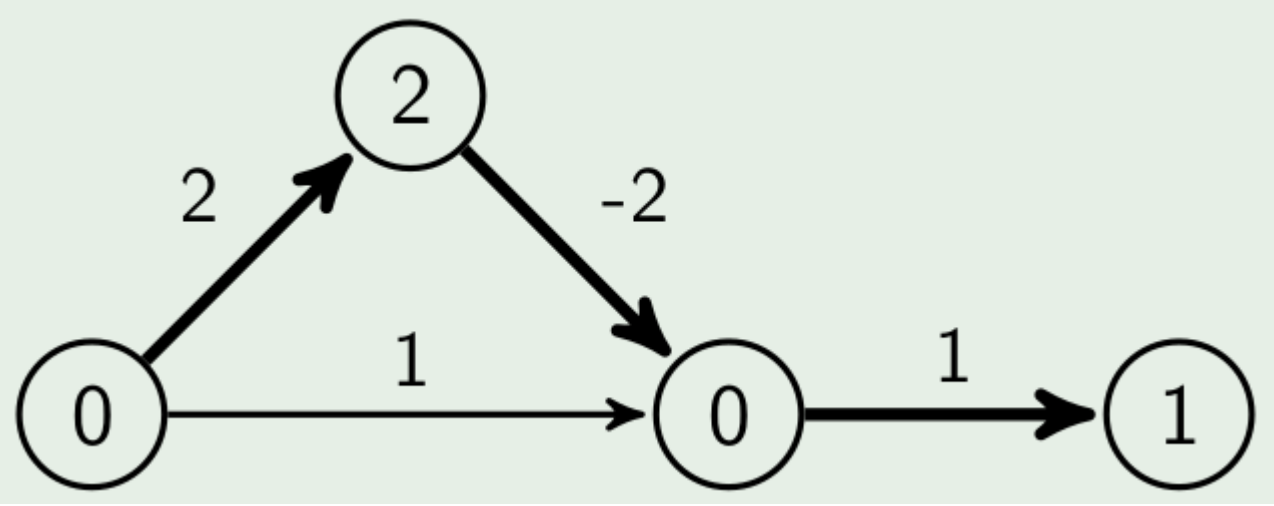
Por hipótese de contradição, assumimos que $\backslash(\text{dist}(u) < d[u]\backslash)$.
Portanto, $\backslash(d[y] = \text{dist}(y) \leq \text{dist}(u) < d[u]\backslash)$. Falso, pois $\backslash(u\backslash)$ tem o menor $\backslash(d\backslash)$ dentre os vértices fora de $\backslash(S\backslash)$.

A suposição de que os pesos são não negativos foi utilizada na demonstração de corretude. De fato, o exemplo abai: que o algoritmo de Dijkstra pode não funcionar em grafos com pesos negativos, mesmo quando não existe ciclo ne

Solução do algoritmo de Dijkstra:



Solução ótima:



Caminhos mínimos em grafos com pesos negativos, mas sem ciclos negativos: Algoritmo Bellman-Ford

Quando o grafo tem ciclo de custo negativo, a solução ótima é passar por este ciclo infinitas vezes.

Se acrescentarmos a restrição de que o caminho só pode passar no máximo uma vez em cada vértice, isso resulta problema NP-completo.

Vamos então nos restringir a grafos sem ciclo negativo.

O algoritmo de Bellman-Ford permite arestas de peso negativo.

Retorna valor booleano:

- FALSO, se existe ciclo negativo atingível a partir de (s) ; ou
- VERDADEIRO, caso contrário.
Neste caso, $(d[])$ tem as distâncias mínimas de (s) para cada vértice, e $(\pi[])$ fornece a árvore de caminhos mínimos.

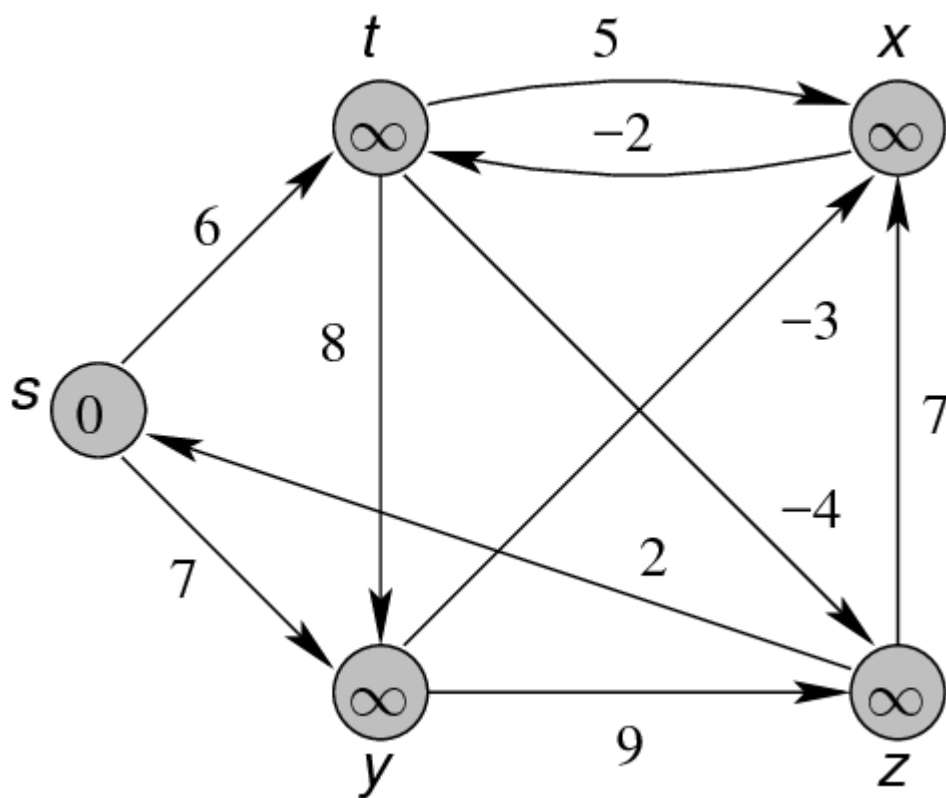
Como não existe ciclo negativo, existe solução ótima que passa no máximo uma vez em cada vértice, pois se passar de uma vez podemos remover o ciclo sem piorar a solução. Portanto, existe caminho mínimo de (s) para qualquer nó utilizando no máximo $(V-1)$ arestas.

O algoritmo de Bellman-Ford relaxa todas as arestas $(V-1)$ vezes. Na 1ª vez que relaxar todas as arestas, irá relaxar arestas de todos os caminhos mínimos. Na 2ª vez irá relaxar a 2ª aresta de todos os caminhos mínimos, assim sucessivamente. Portanto, se o grafo não tiver ciclo negativo, pela Propriedade de Relaxamento de Caminho teremos $d[v] = dist(u)$ para todo vértice (u) .

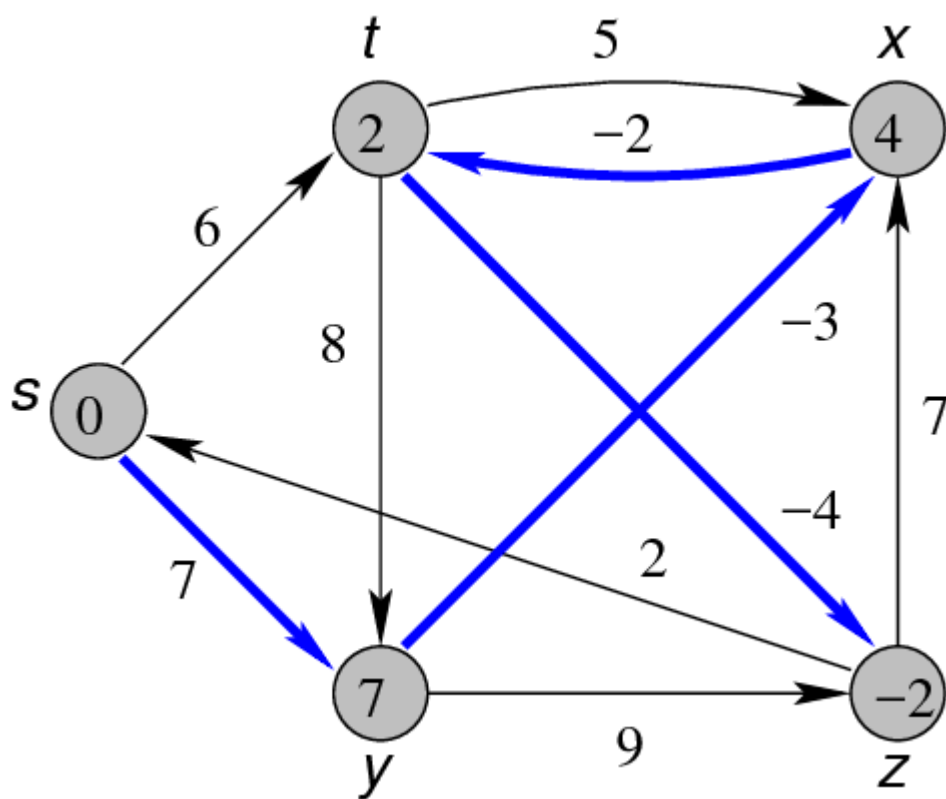
Para testar se o grafo tem ciclo negativo, depois de executar $(V-1)$ vezes a relaxação de todas as arestas, basta verificar se $(d[v] > d[u] + w[u,v])$ para alguma aresta $((u,v))$. Veremos adiante que este teste é suficiente.

```
BELLMAN-FORD(s):  
  INICIALIZAÇÃO(s)  
  relaxe todas as arestas |V|-1 vezes  
  se alguma aresta (u,v) tem d[v] > d[u] + w[u,v]  
    retorne FALSO  
  retorne VERDADEIRO
```

Exemplo: resposta completa nos slides.



Para o exemplo usamos uma ordem fixa de relaxamentos: $((t,x), (t,y), (t,z), (x,y), (y,x), (y,z), (z,x), (z,s), (s,t), (s,y))$.



Corretude:

Caso I: Grafo não tem ciclo negativo atingível de (s) .

Pela Propriedade de Relaxamento de Caminho, vimos que $(d[v] = \text{dist}(v))$ para todo vértice (v) .
Pela Desigualdade Triangular temos que $(d[v] = \text{dist}(v) \leq \text{dist}(u) + w[u,v] = d[u] + w[u,v])$ para toda aresta (u,v) .
Portanto, vai o teste vai retornar VERDADEIRO.

Caso II: Tem ciclo negativo $(C=(v_0,v_1,...,v_k=v_0))$ atingível de (s) .

Como (C) é negativo, temos que $(\sum_{i=1}^k w[v_{i-1},v_i] < 0)$.
Por contradição, assuma que retorna VERDADEIRO. Ou seja, $(d[v_i] \leq d[v_{i-1}] + w[v_{i-1},v_i])$ para todo (i) .
Somando a inequação anterior para toda aresta $((v_{i-1},v_i))$ de (C) : $(\sum_{i=1}^k d[v_i] \leq \sum_{i=1}^k d[v_{i-1}] + \sum_{i=1}^k w[v_{i-1},v_i])$.

Como $(v_k = v_0)$ temos que $(\sum_{i=1}^k d[v_i] = d[v_1] + \dots + d[v_{k-1}] + d[v_k] = \sum_{i=1}^k d[v_i])$.
Portanto $(\sum_{i=1}^k w[v_{i-1},v_i] \geq 0)$ (Falso).

BELLMAN-FORD(s):	
INICIALIZAÇÃO(s)	
para i <- 1 até V -1	O(V)
para cada aresta (u,v)	O(VE)
RELAX(u,v)	O(VE)
para cada aresta (u,v)	O(E)
se d[v] > d[u] + w[u,v]	O(E)
retorne FALSO	O(1)
retorne VERDADEIRO	O(1)

Complexidade: $(O(VE))$.

[◀ Notas de aula: busca em largura \(BFS\)](#)

Seguir para...

[Vídeo: busca em profundidade \(40min\) ▶](#)

©2020 – Universidade Federal do Ceará – Campus Quixadá.

Todos os direitos reservados.

Av. José de Freitas Queiroz, 5003

Cedro – Quixadá – Ceará CEP: 63902-580

Secretaria do Campus: (88) 3411-9422

 [Baixar o aplicativo móvel.](#)